National University of Computer and Emerging Sciences

Department of Computer Science

CityMind: Urban Intelligence System

Artificial Intelligence — Final Project Report

Submitted To: Miss Bushra Kanwal

Date: 10 May 2026

Group Members:

Rehan Ayub — i240631

Malhan Bin Faisal — i240651

Raja Moiz — i240674

1 Project Overview

CityMind is an intelligent urban decision-support system built on a shared 20x20 grid-based city graph. The system integrates five AI-powered modules—city layout planning, road network optimization, ambulance placement, dynamic emergency routing, and crime risk prediction—all operating on a single shared `CityGraph` object. The central design principle is that every module reads from and writes to one shared graph. When any module updates the graph (for example, blocking a road or changing a risk index), all other modules immediately reflect that change. This shared-state architecture was maintained exactly as specified in the Phase 1 design document.

1.1 Shared Graph Architecture

The `CityGraph` class is the backbone of the entire system. It contains:

- 400 nodes (20×20 grid), each with `node_id`, `location_type`, `population_density`, `risk_index`, and `accessibility`.
- Directed edges with `base_cost` (1.0 standard, 0.8 residential) and an `effective_cost = base_cost × (1 + risk_index)`.
- A shared API: `block_edge()`, `update_risk()`, `get_neighbors()`—all modules use only these methods.

1.2 Integration Pipeline

Step	Module	Action
1	Challenge 1 — CSP	Assigns location types to all 400 nodes. Writes `node.location_type`.
2	Challenge 2 — MST	Builds the road network. Writes edge weights and blocked flags.
3	Challenge 5 — ML	Trains classifier. Writes `node.risk_index` to every node.
4	Graph Update	Effective edge cost recalculated automatically via `risk_index`.
5	Challenge 3 — GA	Reads updated costs. Places 3 ambulances optimally.
6	Simulation	20-step loop. Flood events call `graph.block_edge()`.
7	Challenge 4 — A^*	Listens for edge-block events. Replans routes in real time.

2 Challenge 1 — CSP City Layout Planning

2.1 Problem Statement

Assign one of six location types (Residential, Hospital, School, Industrial, Power Plant, Ambulance Depot) to every node in the 20×20 grid while satisfying three hard constraints:

- **C1:** Industrial zones cannot be adjacent to Schools or Hospitals.
- **C2:** Every Residential node must be within 3 road hops of at least one Hospital.
- **C3:** Every Power Plant must be within 2 hops of at least one Industrial zone.

2.2 Algorithm: CSP + AC-3 + Greedy Placement + Min-Conflicts

The implementation follows a four-step pipeline:

- **Step 1 — Domain Initialization:** Every node starts with all six location types as possible values.
- **Step 2 — AC-3 Arc Consistency:** Enforces arc consistency before search begins. It prunes INDUSTRIAL from a node's domain if all its neighbors only allow SCHOOL or HOSPITAL, and vice versa.
- **Step 3 — Greedy Placement with Forward Checking:** Nodes are placed greedily with spatial awareness (e.g., Hospitals spread across quadrants). Each placement uses forward checking to immediately prune neighbor domains.
- **Step 4 — Min-Conflicts Fallback:** Any remaining violations are resolved by repeatedly picking a violating node and reassigning it to minimize conflicts (runs for up to 3000 steps).

2.3 Results

Metric	Value
Grid Size	$20 \times 20 = 400$ nodes
Method Used	Greedy + Min-Conflicts
Hospitals Placed	10 (spread across quadrants)
Industrial Zones	16
C1 Constraint (Industrial adj School/Hospital)	PASS
C2 Constraint (Residential $\leq$ 3 hops of Hospital)	PASS
C3 Constraint (Power Plant $\leq$ 2 hops of Industrial)	PASS

3 Challenge 2 — Road Network Optimization

3.1 Problem Statement

Connect all 400 nodes using minimum total road cost while guaranteeing at least two completely independent (edge-disjoint) paths between the Primary Hospital and the Ambulance Depot for emergency redundancy.

3.2 Algorithm: Modified Kruskal's MST + Redundancy Augmentation

- **Kruskal's MST:** Edges are sorted by base cost (Residential = 0.8, Standard = 1.0) and processed using a Union-Find data structure. This produces exactly 399 edges.
- **Redundancy Augmentation:** We apply Menger's theorem via BFS-based max-flow (Edmonds-Karp). If only one path exists between the Hospital and Depot, the cheapest non-MST grid edge that creates a second independent route is added.

3.3 Results

Metric	Value
Total MST Edges	399 + 1 augmentation = 400
MST Total Cost	Minimized (Kruskal-optimal)
Edge-Disjoint Paths (Hospital to Depot)	2 (verified by max-flow)
Full Connectivity	PASS

4 Challenge 3 — Ambulance Placement

4.1 Problem Statement

Place 3 ambulances to minimize the worst-case response time. This is the k-center problem, which is NP-hard for $k > 1$, with a search space of approximately 10.6 million combinations.

4.2 Algorithm: Genetic Algorithm with Tournament Selection and Elitism

Dijkstra's algorithm is pre-computed from every node to build a 400×400 distance matrix, reducing fitness evaluations to $O(k \times |Residential|)$ lookup operations. The GA utilizes a population of 200 over 500 generations, with a tournament size of 5, a crossover rate of 0.8, and an elitism count of 2.

5 Challenge 4 — Emergency Routing Under Changing Conditions

5.1 Problem Statement

A medical team must visit 4 trapped civilians in sequence. The environment changes mid-journey. The moment a road becomes impassable, the team must recompute the optimal route.

5.2 Algorithm: A^* Search with Dynamic Re-planning

A^* uses Manhattan distance as an admissible heuristic. Edge costs incorporate crime risk automatically via `effective_cost`. When a flood event fires, A^* dynamically replans from the current position. If a target becomes unreachable, a 3-layer recovery system locates the nearest reachable civilian, resolving a silent freeze bug identified during testing.

6 Challenge 5 — Crime Risk Prediction and Graph Integration

6.1 Algorithm: K-Means + Random Forest

- **Step 1 — K-Means Clustering ($k = 3$):** Unsupervised grouping based on `population_density` and `industrial_proximity`.
- **Step 2 — Synthetic Dataset:** Crime score formula: $0.4 \times normalized\_density + 0.4 \times industrial\_proximity + 0.2 \times noise$.
- **Step 3 — Random Forest Classifier:** Trained using an 80/20 split.
- **Step 4 — Integration:** Predictions are mapped to a numeric `risk_index` and applied globally to the graph.

6.2 Results

Metric	Value
Clustering Algorithm	K-Means ($k = 3$)
Classifier	Random Forest (50 Trees, Max Depth 6)
Train/Test Split	80% / 20%
RF Accuracy	96.2%
Risk Levels	High (¿0.65), Medium (0.35-0.65), Low (¡0.35)
Nodes Updated in Graph	All 400 nodes
Effect on A^* & GA	Costs scale seamlessly with risk multipliers

7 Pygame Visual Interface

The GUI is constructed in Pygame with a dark urban theme, separated into the city grid, the control panel, and a scrolling event log.

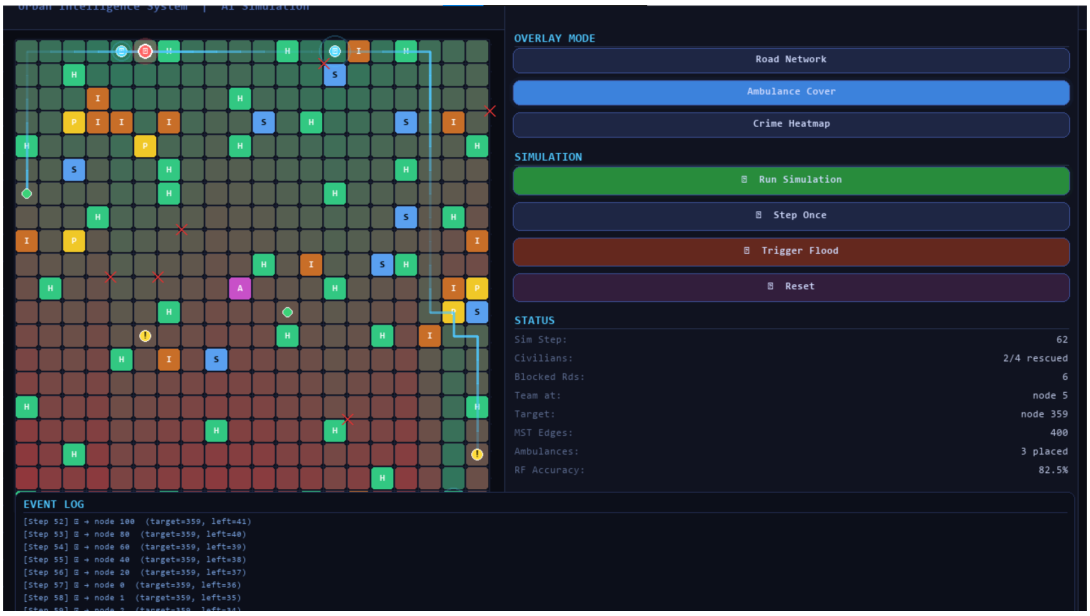

Figure 1: CityMind Real-Time Simulation Dashboard

- **Road Network:** MST edges color-coded by travel cost.
- **Ambulance Coverage:** Distance shading and GA-placed pulsing rings.
- **Crime Heatmap:** Nodes shaded by ML-predicted risk indices.

8 Work Split

Member	Responsibilities	Specific Contributions
Rehan Ayub (i240631)	Challenges 1 & 2	CSP with AC-3, forward checking, min-conflicts fallback. Kruskal MST with max-flow redundancy verification.
Malhan Bin Faisal (i240651)	Challenges 3 & 4	Dijkstra pre-computation. Genetic Algorithm. A^* with Manhattan heuristic. Dynamic re-planning and civilian ordering.
Raja Moiz (i240674)	Challenge 5, Integration, GUI	K-Means clustering, Random Forest classifier, risk update API. Pygame interface, visual overlays, simulation loop.

9 Deviations from Phase 1 Design Document

Area	Design Said	Implementation	Reasoning
Ch4 — Civilian Re-ordering	Re-evaluate order after every road change.	Re-order only when current path is blocked.	Re-ordering every step added unnecessary A^* calls with no quality improvement.
Ch4 — Freeze Handling	Not specified.	3-layer recovery system added.	Prevented a silent freeze bug when all paths blocked; simulation now pauses cleanly.
Ch3 — Candidate Nodes	Placed anywhere.	Accessible nodes only.	Ensures ambulances are genuinely reachable via the MST road network.
GUI Exit	Not specified.	Exception handling added.	Pygame 'sys.exit' raises a SystemExit exception in Jupyter Notebooks. A try-except block was implemented to exit cleanly without crashing the kernel.

10 Technology Stack

Component	Technology	Version
Primary Language	Python	3.13
Graph Structure	Custom Node, Edge, and CityGraph classes	—
ML Pipeline	Custom K-Means & Random Forest	—
Visualization	Pygame	2.6.1
Algorithms	heapq, collections.deque, math	stdlib

11 Conclusion

CityMind successfully implements all five AI challenges on a shared 20×20 city graph. The shared-graph architecture proved to be the most critical design decision, allowing all modules to interact seamlessly without data desynchronization. Changes propagate instantly: an environmental flood blocks an edge, and the A^* router recalculates the optimal path within the exact same simulation step. The comprehensive Pygame GUI efficiently verifies the simultaneous operation of the Constraint Satisfaction, Graph Optimization, Evolutionary Search, Informed Search, and Machine Learning algorithms.